Comparando Funções Hash

Resolução de Problemas Estruturados em Computação | 4ºB BES

Daniel de Castro Felix | 40109952
Mateus Canatto Campos | 40111563
Murilo Caxambu Monteiro | 40113745

05/11/2026

IMPLEMENTAÇÃO

Para as funções *hashDivisao*, *hashPrimo*, *hashDobra*, *hashMeioQuadrado*, *aplicaHash*, *nomeFuncao*, *nomeConjunto*, *rehash*, *criaTabela*, *insere* e *busca*, a realização foi muito intuitiva de se realizar, necessitando apenas seguir os passos da referência disponível pelo professor: o *Trabalho 2- Comparando Funcoes Hash*.

Entretanto, a partir da função *geraConjunto* as funcionalidades começaram a ficar mais complexas em sua lógica. Foi necessário percorrer o vetor a cada número sorteado e verificar se a chave já existia, descartando as duplicadas.

Para a função *executaCombinacao*, a implementação foi complicada, pois era necessário conhecimentos além dos disponíveis passo-a-passo, exigindo um maior raciocínio. Uma das decisões tomadas foi a nomeação de alguns atributos, pois haviam atributos que deveriam representar um atributo com um nome mais específico, como o *usadas* x *posicoesUsadas*. Além disso, para o *String.format*, foi preciso uma rápida pesquisa disponível das referências para compreender a forma de como o Java aceita a sintaxe.

Para a função *executaTestes* os primeiros testes de código “H” foi o maior desafio dessa função, necessitando compreender como seguir com as referências, mas isso foi resolvido de forma simples criando um vetor que armazenaria a tabela. Enquanto que para os outros testes, o passo a passo da referência foi mais que o suficiente para escrever quase que fluidamente o código. Necessitando apenas pesquisar pequenas sintaxes e verificar atributos e retornos nas outras partes do código.

Para a main, a implementação foi até que bem fácil. Foi seguido exatamente como estava descrito no documento. Para algumas coisas, como o PrintWriter e o Random foi necessário pesquisar para saber como funcionava para implementar no código, de resto foi basicamente seguir com o roteiro. No fim foi retornado o valor esperado no console: “TESTES: 17 de 17 aprovados”

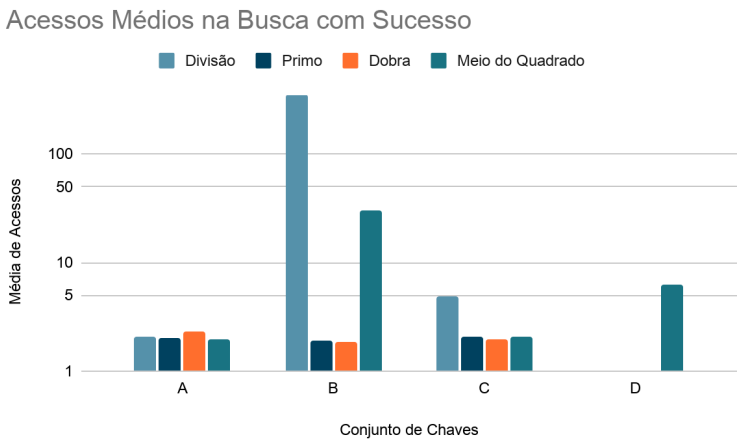
TABELA DE RESULTADOS

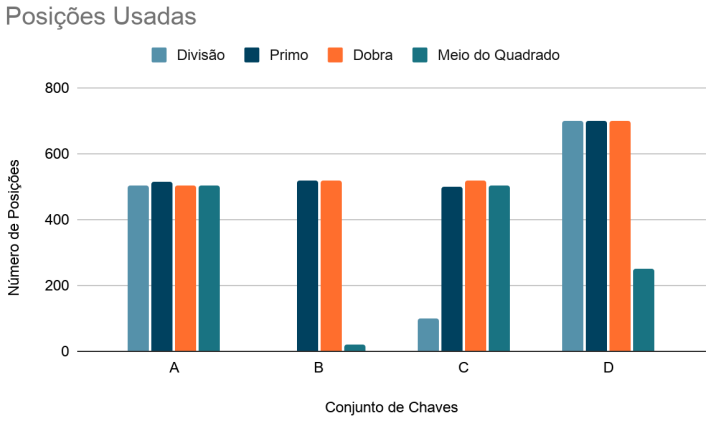
conjunto	funcao	posicoesUsadas	maiorConcetracao	somaQuadrados	colisoes	acessosInsecaoTotal	maiorAcessosUmaInsercao	acessosBusca
A	divisao	502	5	1212	253	1439	17	2,06

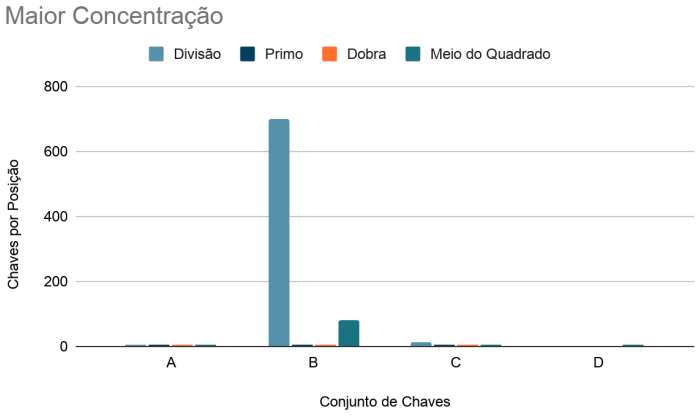
A	primo	515	4	1150	240	1413	26	2,02
A	dobra	502	5	1198	250	1641	40	2,34
A	meioQuadrado	504	5	1204	238	1377	17	1,97
B	divisao	1	700	490000	699	245350	700	350,50
B	primo	520	4	1136	216	1325	27	1,89
B	dobra	518	4	1120	233	1302	20	1,86
B	meioQuadrado	22	82	27118	678	21291	225	30,42
C	divisao	100	13	5740	600	3425	36	4,89
C	primo	501	6	1210	245	1459	29	2,08
C	dobra	517	5	1154	226	1393	35	1,99
C	meioQuadrado	503	5	1208	242	1447	40	2,07
D	divisao	700	1	700	0	700	1	1,00
D	primo	700	1	700	0	700	1	1,00
D	dobra	700	1	700	0	700	1	1,00

D	meioQuadrado	250	7	2822	650	4392	10	6,27

GRÁFICOS DE BARRAS AGRUPADAS







RESPOSTAS ÀS PERGUNTAS

1. No conjunto A (aleatório), as quatro funções tiveram desempenho parecido? Qual teve menos colisões e por que a diferença entre elas é pequena?
R: Sim, no conjunto todas as funções mostraram um resultado semelhante. A função com menor número de colisões foi a meioQuadrado com 238. Isso acontece porque as chaves do conjunto são geradas de maneira aleatória, sem nenhum padrão, fazendo com que elas tenham a mesma probabilidade de ocorrer e se distribuem de maneira semelhante.

2. No conjunto B, o que aconteceu com a função divisão? Explique por que posicoesUsadas foi 1 e quantos acessos gastou a inserção da última chave.
R: A função divisão teve o maior número de colisões(699) e média de 350,5 acessos. Todas as chaves do conjunto B são múltiplos de 1000, com o cálculo sempre sendo igual a 0, de forma que a variável posicoesUsadas sempre seja 1. Assim, toda vez que uma nova inserção é realizada ela tem que percorrer todo o bloco do início, gastando 700 acessos para a última chave.

3. Ainda no conjunto B, por que a função primo se saiu tão melhor que a divisão? Relacione com o fato de 1000 = $2^3 \times 5^3$ e de 997 ser primo.
R: Como 1000 = $2^3 \times 5^3$ e as chaves são múltiplas de 1000 a divisão sempre vai ser exata. Já o 997 é primo e não tem fatores em comum com 1000, assim cada divisão produz restos diferentes para cada múltiplo de 1000, e as chaves ficam espalhadas.

4. No conjunto C, por que a função divisão usa apenas 100 posições? Que fração da tabela ficou sem nenhuma chave vinda da função hash?
R: Como as chaves nesse conjunto são múltiplas de 10 e terminam em 0 o resto da divisão gera apenas 100 números. Desta forma, cerca de 90% (9/10) da tabela ficam vazias antes do rehashing, com 900 posições não usadas.

5. No conjunto D (sequencial), a divisão, o primo e a dobra não tiveram nenhuma colisão. Explique por quê. Por que o meio do quadrado foi pior? Cite posicoesUsadas e acessosBuscaSucessoMedia.
R: As chaves do conjunto D são sequenciais, somar 1 na chave avança o índice em +1 nas funções divisão, primo e dobra, preenchendo 700 posições sem nenhuma colisão (acessosBuscaSucessoMedia = 1,00). Já no meioQuadrado, elevar números consecutivos ao quadrado faz com que os dígitos variem e se repitam, derrubando posicoesUsadas para 250 e aumentando acessosBuscaSucessoMedia para 6,27.

6. Explique por que posicoesNuncaUsadas da função primo é sempre pelo menos 3.
R: Na função o resto sempre varia entre 0 e 996. Como a tabela tem 1000 posições, os índices 997, 998 e 999 nunca podem ser gerados diretamente pela função hash (apenas por rehashing) assim posicoesNuncaUsadas é sempre pelo menos 3.

7. Mostre, com a conta, por que a função do meio do quadrado precisa de long para a chave 9999999. Qual é o maior valor que cabe em um int em Java?
R: O maior valor que cabe em um int é 2.147.483.647. Para a chave 9999999, o seu quadrado é 9.999.999 x 9.999.999 = 99.999.980.000.001, o que ultrapassa o limite do int, gerando um resultado errado. O long suporta esse número e permite extrair o índice correto (800).
8. Em qual conjunto foi maior a diferença entre a melhor e a pior função? Compare com

acessosBuscaSucessoMedia e com somaQuadrados e explique o que somaQuadrados mede.
R: No conjunto B, onde a pior função foi divisao (acessosBuscaSucessoMedia = 350,50 e somaQuadrados = 490000) e a melhor foi dobra (acessosBuscaSucessoMedia = 1,86 e somaQuadrados = 1120). A métrica somaQuadrados soma o quadrado da quantidade de chaves de cada posição, mede o grau de concentração e desuniformidade.

9. Por que maiorAcessosUmaInsercao pode ser maior que maiorConcentracao? Cite uma linha do CSV em que isso acontece.
R: maiorConcentracao tem apenas chaves que caíram no mesmo índice inicial, enquanto maiorAcessosUmaInsercao soma todo o caminho do rehashing linear. Quando posições vizinhas já estão ocupadas, a inserção salta todo o bloco adjacente, como ocorre na linha A;dobra (concentração 5 e 40 acessos).
10. Se os códigos dos produtos do comerciante dos slides terminassem sempre em 00 (por exemplo, 1234500), qual função o grupo recomendaria e por quê?
R: Recomendamos primo ou dobra. Códigos terminados em 00 são múltiplos de 100, fazendo divisão usar apenas 10 posições e meioQuadrado repetir padrões pelos zeros finais. Já primo (997 não divide 100) e dobra mantêm boa distribuição.

CONCLUSÃO

Nós escolhemos a função hash primo como padrão, pois ela é uma função simples de implementar. Os resultados dos experimentos mostraram que a função por primo gerou a menor quantidade de colisões, sendo 701, próximo a dobra com 709, a qual se mostra um pouco mais complexa de implementar. Por isso, a escolha foi a função com menor colisão, concedendo maior distribuição das chaves na tabela.

DECLARAÇÃO

"Declaramos que o código e o texto deste trabalho foram produzidos pelos integrantes do grupo, sem o uso de ferramenta de inteligência artificial generativa e sem cópia de outros grupos ou de fontes externas."

- Daniel de Castro Felix 40109952
- Mateus Canatto Campos 40111563
- Murilo Caxambu Monteiro 40113745

REFERÊNCIAS

Documento disponibilizado no AVA: Trabalho 2 - Comparando Funcoes Hash.pdf
https://www.w3schools.com/java/ref_string_format.asp
https://codegym.cc/groups/posts/java-printwriter-class